

I. TÌM HIỂU 3 PUBLIC API

1. GitHub REST API

- Loại API: REST API
- Base URL: <https://api.github.com>
- Authentication:
 - Personal Access Token (PAT)
 - OAuth 2.0
- Versioning:
 - Có versioning thông qua header API version
- Resource Identifier:
 - Sử dụng cả số nguyên (id) và chuỗi định danh (owner/repository name)
- Ví dụ Resource

Lấy thông tin repository: GET /repos/octocat/Hello-World

Nhận xét:

GitHub API là một REST API tiêu biểu. Tài nguyên được biểu diễn dưới dạng resource như users, repositories, issues, pull requests. API sử dụng JSON làm định dạng dữ liệu và tuân theo mô hình Client-Server

2. OpenWeather API

- Loại API: REST API
- Base URL: <https://api.openweathermap.org>
 - Authentication: API key
 - Versioning: Có versioning (v2.5, v3.0)
 - Resource Identifier: City ID hoặc tọa độ địa lý (latitude, longitude)
- Ví dụ Resource

Lấy thời tiết theo thành phố: GET /data/2.5/weather?q=Hanoi

Nhận xét:

OpenWeather API cung cấp dữ liệu thời tiết theo thời gian thực. API phù hợp cho ứng dụng web, mobile hoặc IoT cần thông tin thời tiết.

3. Spotify Web API

- Loại API: REST API
- Base URL: <https://api.spotify.com/v1>
- Authentication: OAuth 2.0
- Versioning: Có versioning thông qua đường dẫn /v1
- Resource Identifier: Spotify ID (chuỗi định danh duy nhất)
- Ví dụ Resource

Lấy thông tin một bài hát: GET /tracks/{id}

Nhận xét:

Spotify API được thiết kế theo hướng resource-centric. Các resource phổ biến gồm artists, albums, playlists và tracks.

II. SO SÁNH VỚI KIẾN THỨC TRONG GIÁO TRÌNH

1. Resource-Oriented Design

- Hai tác giả nhấn mạnh việc thiết kế API xoay quanh resource thay vì function.

Ví dụ: GET /books/10 tốt hơn GET /getBook?id=10

2. Tính nhất quán (Consistency)

- Các endpoint cần tuân theo cùng một quy tắc đặt tên, cùng kiểu phản hồi và cùng cách xử lý lỗi.

Ví dụ: /users, /books, /orders thay vì /getUser, /createBook, /deleteOrder

3. Khả năng khám phá (Discoverability)

- API nên dễ học và dễ sử dụng.
- Người dùng chỉ cần đọc tài liệu API là có thể hiểu: endpoint nào tồn tại, request như thế nào, response ra sao, lỗi nào có thể xảy ra

4. Tính mở rộng (Extensibility)

- API cần được thiết kế để phát triển trong tương lai mà không phá vỡ client cũ.

Ví dụ: /v1/books, /v2/books

5. Tài liệu hóa tốt

- Một API tốt luôn đi kèm tài liệu mô tả rõ: endpoint, method, request body, response body, status code

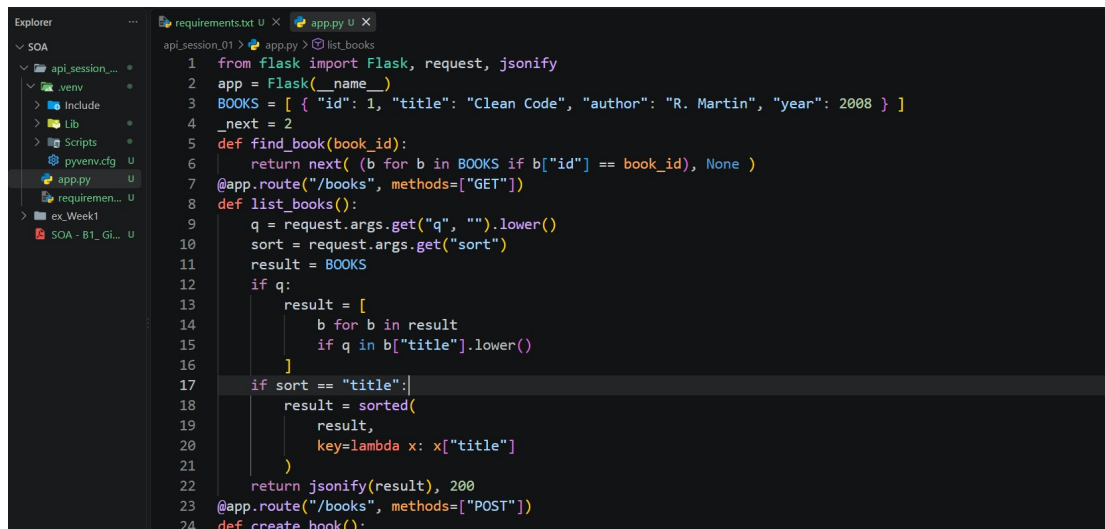
Ví dụ: OpenAPI, Swagger

III. MỞ RỘNG BÀI 6 FLASK CRUD

Bổ sung: Search: GET /books?q=..., Sort: GET /books?sort=title,

Validate year >= 1900

Code:



```
1 from flask import Flask, request, jsonify
2 app = Flask(__name__)
3 BOOKS = [ { "id": 1, "title": "Clean Code", "author": "R. Martin", "year": 2008 } ]
4 _next = 2
5 def find_book(book_id):
6     return next( (b for b in BOOKS if b["id"] == book_id), None )
7 @app.route("/books", methods=["GET"])
8 def list_books():
9     q = request.args.get("q", "").lower()
10    sort = request.args.get("sort")
11    result = BOOKS
12    if q:
13        result = [
14            b for b in result
15            if q in b["title"].lower()
16        ]
17    if sort == "title":
18        result = sorted(
19            result,
20            key=lambda x: x["title"]
21        )
22    return jsonify(result), 200
23 @app.route("/books", methods=["POST"])
24 def create_book():
```

```

def create_book():
    global _next
    body = request.get_json(silent=True) or {}
    title = body.get("title")
    author = body.get("author")
    year = body.get("year")
    if not title or not author:
        return { "error": "title and author required" }, 400
    if not isinstance(year, int):
        return { "error": "year must be integer" }, 400
    if year < 1900:
        return { "error": "year must be >= 1900" }, 400
    book = {
        "id": _next, "title": title, "author": author, "year": year }
    _next += 1
    BOOKS.append(book)
    return jsonify(book), 201
if __name__ == "__main__":
    app.run(debug=True)

```

Output Debug Console **Terminal** Ports

```

TOP-HCT3QG47 MINGW64 /e/SOA (main)
THI@LAPTOP-HCT3QG47 MINGW64 /e/SOA (main)
$ curl -i http://localhost:5000/books?q=clean
HTTP/1.1 200 OK
Server: Werkzeug/3.1.8 Python/3.12.3
Date: Sun, 13 Sep 2026 13:56:23 GMT
Content-Type: application/json
Content-Length: 96
Connection: close

```

```

[
  {
    "author": "R. Martin",
    "id": 1,
    "title": "Clean Code",
    "year": 2008
  }
]

```

```

THI@LAPTOP-HCT3QG47 MINGW64 /e/SOA (main)
$ curl -i http://localhost:5000/books?sort=title
HTTP/1.1 200 OK
Server: Werkzeug/3.1.8 Python/3.12.3
Date: Sun, 13 Sep 2026 13:57:04 GMT
Content-Type: application/json
Content-Length: 96
Connection: close

```

```

[
  {
    "author": "R. Martin",
    "id": 1,
    "title": "Clean Code",
    "year": 2008
  }
]

```